

```
[1]: # =====  
#           STEP 6 – VISUALIZATION & DASHBOARD  
# =====  
  
# -----  
# STEP 1 – IMPORT LIBRARIES  
# -----  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
from pathlib import Path  
  
# Display settings  
pd.set_option("display.max_columns", None)  
pd.set_option("display.max_rows", 100)  
  
# -----  
# STEP 2 – LOAD REQUIRED DATASETS  
# -----  
  
transactions = pd.read_csv("ecommerce_cleaned_transactions.csv")  
customers = pd.read_csv("customer_level_features.csv")  
rfm = pd.read_csv("customer_rfm_segments.csv")
```

```
# -----  
# STEP 3 – CONVERT DATE COLUMN  
# -----
```

```
transactions["purchase_date"] = pd.to_datetime(  
    transactions["purchase_date"],  
    errors="coerce"  
)
```

```
# -----  
# STEP 4 – BASIC DATA CHECK  
# -----
```

```
print("Transactions Shape:", transactions.shape)  
print("Customer-Level Shape:", customers.shape)  
print("RFM Shape:", rfm.shape)
```

```
print("\nTransaction Columns:")  
print(transactions.columns.tolist())
```

```
print("\nCustomer Columns:")  
print(customers.columns.tolist())
```

```
print("\nRFM Columns:")  
print(rfm.columns.tolist())
```

```
# =====  
# SECTION A – OVERALL BUSINESS KPIs  
# =====  
  
# -----  
# STEP 5 – CALCULATE KEY BUSINESS KPIs  
# -----  
  
total_transactions = len(transactions)  
  
total_customers = transactions["customer_id"].nunique()  
  
total_revenue = transactions["total_purchase_amount"].sum()  
  
average_order_value = transactions["total_purchase_amount"].mean()  
  
total_quantity = transactions["quantity"].sum()  
  
average_quantity = transactions["quantity"].mean()  
  
if "returns" in transactions.columns:  
    total_returns = transactions["returns"].sum()  
else:  
    total_returns = 0  
  
if "churn" in transactions.columns:  
    churn_rate = transactions["churn"].mean() * 100  
else:  
    churn_rate = np.nan
```

```
# STEP 6 - DISPLAY KPIS
```

```
# -----
```

```
print("\n===== BUSINESS KPIS =====\n")
```

```
print("Total Transactions      :", total_transactions)
print("Total Customers        :", total_customers)
print("Total Revenue           :", round(total_revenue, 2))
print("Average Order Value     :", round(average_order_value, 2))
print("Total Quantity Sold      :", total_quantity)
print("Average Quantity/Order   :", round(average_quantity, 2))
print("Total Returns            :", total_returns)
print("Overall Churn Rate (%)    :", round(churn_rate, 2))
```

```
# =====
```

```
# SECTION B - SALES TREND VISUALIZATION
```

```
# =====
```

```
# -----
```

```
# STEP 7 - MONTHLY SALES
```

```
# -----
```

```
monthly_sales = (
    transactions
    .set_index("purchase_date")
    .resample("M")["total_purchase_amount"]
    .sum()
    .reset_index()
```

```
reset_index(),  
)  
  
monthly_sales["Month"] = monthly_sales["purchase_date"].dt.strftime("%Y-%m")  
  
# -----  
# STEP 8 - MONTHLY REVENUE LINE CHART  
# -----  
  
plt.figure(figsize=(6,3))  
  
plt.plot(  
    monthly_sales["purchase_date"],  
    monthly_sales["total_purchase_amount"],  
    marker="o"  
)  
  
plt.title("Monthly Revenue Trend")  
plt.xlabel("Month")  
plt.ylabel("Total Revenue")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```

```
# -----  
# STEP 9 – YEARLY SALES  
# -----
```

```
yearly_sales = (  
    transactions  
    .assign(Year=transactions["purchase_date"].dt.year)  
    .groupby("Year", as_index=False)["total_purchase_amount"]  
    .sum()  
)
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  
    data=yearly_sales,  
    x="Year",  
    y="total_purchase_amount"  
)
```

```
plt.title("Yearly Revenue")  
plt.xlabel("Year")  
plt.ylabel("Total Revenue")  
plt.tight_layout()  
plt.show()
```

```
# =====  
# SECTION C – PRODUCT CATEGORY VISUALIZATION  
# =====
```

```
# -----  
# STEP 10 – CATEGORY PERFORMANCE  
# -----
```

```
category_summary = (  
    transactions  
    .groupby("product_category")  
    .agg(  
        Revenue=("total_purchase_amount", "sum"),  
        Transactions=("customer_id", "count"),  
        Quantity=("quantity", "sum"),  
        Average_Order_Value=("total_purchase_amount", "mean")  
    )  
    .reset_index()  
    .sort_values("Revenue", ascending=False)  
)  
  
print("\nCategory Performance:")  
display(category_summary)
```

```
# -----  
# STEP 11 – CATEGORY REVENUE CHART  
# -----
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  
    data=category_summary,  
    x="product_category",  
    y="Revenue"  
)
```

```
plt.title("Revenue by Product Category")  
plt.xlabel("Product Category")  
plt.ylabel("Revenue")  
plt.xticks(rotation=30)  
plt.tight_layout()  
plt.show()
```

```
# -----  
# STEP 12 – CATEGORY TRANSACTION DISTRIBUTION  
# -----  
  
plt.figure(figsize=(6,3))  
  
sns.barplot(  
    data=category_summary,  
    x="product_category",  
    y="Transactions"  
)  
  
plt.title("Transactions by Product Category")  
plt.xlabel("Product Category")  
plt.ylabel("Number of Transactions")  
plt.xticks(rotation=30)  
plt.tight_layout()  
plt.show()
```

```
# =====  
# SECTION D – PAYMENT METHOD ANALYSIS  
# =====  
  
# -----  
# STEP 13 – PAYMENT METHOD SUMMARY  
# -----  
  
payment_summary = (  
    transactions  
    .groupby("payment_method")  
    .agg(  
        Transactions=("customer_id", "count"),  
        Revenue=("total_purchase_amount", "sum"),  
        Average_Order_Value=("total_purchase_amount", "mean")  
    )  
    .reset_index()  
    .sort_values("Revenue", ascending=False)  
)  
  
print("\nPayment Method Performance:")  
display(payment_summary)
```

```
# -----  
# STEP 14 - PAYMENT METHOD CHART  
# -----
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  
    data=payment_summary,  
    x="payment_method",  
    y="Transactions"  
)
```

```
plt.title("Transactions by Payment Method")  
plt.xlabel("Payment Method")  
plt.ylabel("Number of Transactions")  
plt.xticks(rotation=30)  
plt.tight_layout()  
plt.show()
```

```
# =====  
# SECTION E – CUSTOMER SEGMENTATION VISUALIZATION  
# =====
```

```
# -----  
# STEP 15 – CUSTOMER SEGMENT SUMMARY  
# -----
```

```
if "Customer_Segment" in rfm.columns:  
  
    segment_summary = (  
        rfm  
        .groupby("Customer_Segment")  
        .agg(  
            Customers=("customer_id", "count"),  
            Revenue=("monetary", "sum"),  
            Average_Spend=("monetary", "mean"),  
            Average_Frequency=("frequency", "mean"),  
            Average_Recency=("recency", "mean")  
        )  
        .reset_index()  
        .sort_values("Revenue", ascending=False)  
    )  
  
    print("\nCustomer Segment Summary:")  
    display(segment_summary)  
  
else:
```

```
segment_summary = pd.DataFrame()
```

```
# -----  
# STEP 16 - CUSTOMER SEGMENT DISTRIBUTION  
# -----
```

```
if not segment_summary.empty:
```

```
    plt.figure(figsize=(6,3))
```

```
    sns.barplot(  
        data=segment_summary.sort_values(  
            "Customers",  
            ascending=False  
        ),  
        x="Customer_Segment",  
        y="Customers"  
    )
```

```
    plt.title("Customer Distribution by Segment")  
    plt.xlabel("Customer Segment")  
    plt.ylabel("Number of Customers")  
    plt.xticks(rotation=45, ha="right")  
    plt.tight_layout()  
    plt.show()
```

```
# -----  
# STEP 17 – SEGMENT REVENUE  
# -----
```

```
if not segment_summary.empty:
```

```
    plt.figure(figsize=(8,4))
```

```
    sns.barplot(  
        data=segment_summary.sort_values(  
            "Revenue",  
            ascending=False  
        ),  
        x="Customer_Segment",  
        y="Revenue"  
    )
```

```
    plt.title("Revenue Contribution by Customer Segment")  
    plt.xlabel("Customer Segment")  
    plt.ylabel("Revenue")  
    plt.xticks(rotation=45, ha="right")  
    plt.tight_layout()  
    plt.show()
```

```
# =====  
# SECTION F – RFM VISUALIZATION  
# =====
```

```
# -----  
# STEP 18 – RFM SCORE DISTRIBUTION  
# -----
```

```
rfm_score_columns = [  
    "R_Score",  
    "F_Score",  
    "M_Score",  
    "RFM_Score"  
]  
  
available_rfm_scores = [  
    col for col in rfm_score_columns  
    if col in rfm.columns  
]  
  
for col in available_rfm_scores:  
  
    plt.figure(figsize=(6,3))  
  
    sns.countplot(  
        data=rfm,  
        x=col  
    )
```

```
plt.title(f"Distribution of {col}")
plt.xlabel(col)
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

```
# -----
# STEP 19 – RFM SCATTER PLOT
# -----
```

```
if {"recency", "monetary"}.issubset(rfm.columns):
```

```
    plt.figure(figsize=(6,3))
```

```
    sns.scatterplot(
        data=rfm,
        x="recency",
        y="monetary",
        hue="Customer_Segment"
        if "Customer_Segment" in rfm.columns
        else None,
        alpha=0.6
    )
```

```
plt.title("Recency vs Monetary Value")
plt.xlabel("Recency")
plt.ylabel("Monetary Value")
plt.tight_layout()
plt.show()
```

```
# -----
# STEP 20 - FREQUENCY VS MONETARY
# -----
```

```
if {"frequency", "monetary"}.issubset(rfm.columns):
```

```
    plt.figure(figsize=(6,3))
```

```
    sns.scatterplot(
        data=rfm,
        x="frequency",
        y="monetary",
        hue="Customer_Segment"
        if "Customer_Segment" in rfm.columns
        else None,
        alpha=0.6
    )
```

```
plt.title("Frequency vs Monetary Value")
plt.xlabel("Purchase Frequency")
```

```
plt.ylabel("Monetary Value")
plt.tight_layout()
plt.show()
```

```
# =====
```

```
# SECTION G – CHURN VISUALIZATION
```

```
# =====
```

```
# -----
```

```
# STEP 21 – CHURN DISTRIBUTION
```

```
# -----
```

```
if "churn" in rfm.columns:
```

```
    churn_counts = (
        rfm["churn"]
        .value_counts()
        .sort_index()
    )
```

```
    churn_labels = [
        "Active",
        "Churned"
    ]
```

```
plt.figure(figsize=(6,3))

plt.pie(
    churn_counts.values,
    labels=[
        churn_labels[int(x)]
        if int(x) < len(churn_labels)
        else str(x)
        for x in churn_counts.index
    ],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Customer Churn Distribution")
plt.tight_layout()
plt.show()
```

```
# -----
# STEP 22 – CHURN BY CUSTOMER SEGMENT
# -----
```

```
if {
    "Customer_Segment",
```

```

    "churn"
}.issubset(rfm.columns):

segment_churn = (
    rfm
    .groupby("Customer_Segment")
    .agg(
        Customers=("customer_id", "count"),
        Churned_Customers=("churn", "sum"),
        Churn_Rate=("churn", "mean")
    )
    .reset_index()
)

segment_churn["Churn_Rate"] *= 100

plt.figure(figsize=(6,3))

sns.barplot(
    data=segment_churn.sort_values(
        "Churn_Rate",
        ascending=False
    ),
    x="Customer_Segment",
    y="Churn_Rate"
)

```

```
plt.title("Churn Rate by Customer Segment")
plt.xlabel("Customer Segment")
plt.ylabel("Churn Rate (%)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
# =====
# SECTION H – AGE ANALYSIS
# =====
```

```
# -----
# STEP 23 – AGE GROUP CREATION
# -----
```

```
age_column = None
```

```
if "customer_age" in customers.columns:
    age_column = "customer_age"
```

```
elif "customer_age" in rfm.columns:
    age_column = "customer_age"
```

```
elif "Age" in customers.columns:
    age_column = "Age"
```

```
if age_column is not None:

    if "customer_age" not in customers.columns and age_column in rfm.columns:
        age_data = rfm[["customer_id", age_column]].copy()

    else:
        age_data = customers[
            ["customer_id", age_column]
        ].drop_duplicates("customer_id").copy()

age_data["Age_Group"] = pd.cut(
    age_data[age_column],
    bins=[0, 18, 25, 35, 45, 55, 65, 100],
    labels=[
        "Under 18",
        "18-25",
        "26-35",
        "36-45",
        "46-55",
        "56-65",
        "65+"
    ],
    include_lowest=True
)

age_summary = (
    age_data
    .groupby("Age_Group", observed=True)
```

```
.groupby("Age_Group", observed=True)
.size()
.reset_index(name="Customers")
)

plt.figure(figsize=(6,3))

sns.barplot(
    data=age_summary,
    x="Age_Group",
    y="Customers"
)

plt.title("Customer Distribution by Age Group")
plt.xlabel("Age Group")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

```
# =====  
# SECTION I - GENDER ANALYSIS  
# =====
```

```
# -----  
# STEP 24 - GENDER DISTRIBUTION  
# -----
```

```
if "gender" in customers.columns:  
  
    gender_summary = (  
        customers  
        .groupby("gender")  
        .agg(  
            Customers=("customer_id", "nunique"),  
            Revenue=("total_spend", "sum")  
            if "total_spend" in customers.columns  
            else ("customer_id", "count")  
        )  
        .reset_index()  
    )  
  
    print("\nGender Summary:")  
    display(gender_summary)
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  
    data=gender_summary,  
    x="gender",  
    y="Customers"  
)
```

```
plt.title("Customer Distribution by Gender")  
plt.xlabel("Gender")  
plt.ylabel("Number of Customers")  
plt.tight_layout()  
plt.show()
```

```
# =====  
# SECTION J – PURCHASE DAY ANALYSIS  
# =====
```

```
# -----  
# STEP 25 – DAY OF WEEK  
# -----
```

```
transactions["Day_Name"] = transactions[  
    "purchase_date"  
].dt.day_name()
```

```
day_order = [  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday",  
    "Saturday",  
    "Sunday"  
]
```

```
day_summary = (  
    transactions  
    .groupby("Day_Name")  
    .agg(  
        Transactions=("customer_id", "count"),  
        Revenue=("total_purchase_amount", "sum")  
    )  
    .reindex(day_order)  
    .reset_index()  
)
```

```
# -----  
# STEP 26 – DAY OF WEEK CHART  
# -----
```

```
plt.figure(figsize=(6,3))
```

```
sns.barplot(  

```

```

sns.barplot(
    data=day_summary,
    x="Day_Name",
    y="Transactions"
)

plt.title("Purchases by Day of Week")
plt.xlabel("Day")
plt.ylabel("Number of Transactions")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

```

```

# =====
# SECTION K – HOURLY PURCHASE ANALYSIS
# =====

```

```

# -----
# STEP 27 – EXTRACT PURCHASE HOUR
# -----

```

```

transactions["Purchase_Hour"] = (
    transactions["purchase_date"].dt.hour
)

```

```

hour_summary = (

```

```
transactions
.groupby("Purchase_Hour")
.agg(
    Transactions=("customer_id", "count"),
    Revenue=("total_purchase_amount", "sum")
)
.reset_index()
```

```
# -----
# STEP 28 – HOURLY PURCHASE CHART
# -----
```

```
plt.figure(figsize=(6,3))
```

```
sns.lineplot(
    data=hour_summary,
    x="Purchase_Hour",
    y="Transactions",
    marker="o"
)
```

```
plt.title("Purchases by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Transactions")
plt.xticks(range(0, 24))
```

```

plt.xticks(range(0, 24))
plt.tight_layout()
plt.show()

# =====
# SECTION L – QUANTITY ANALYSIS
# =====

# -----
# STEP 29 – QUANTITY DISTRIBUTION
# -----

quantity_summary = (
    transactions
    .groupby("quantity")
    .agg(
        Transactions=("customer_id", "count"),
        Revenue=("total_purchase_amount", "sum")
    )
    .reset_index()
)

plt.figure(figsize=(6,3))

sns.barplot(
    data=quantity_summary,
    x="quantity",

```

```

        y="Transactions"
    )

plt.title("Transaction Distribution by Quantity")
plt.xlabel("Quantity Purchased")
plt.ylabel("Number of Transactions")
plt.tight_layout()
plt.show()

# =====
# SECTION M – RETURN ANALYSIS
# =====

# -----
# STEP 30 – RETURN ANALYSIS
# -----

if "returns" in transactions.columns:

    return_summary = (
        transactions
        .groupby("returns")
        .agg(
            Transactions=("customer_id", "count"),
            Revenue=("total_purchase_amount", "sum")
        )
    )

```

```
        return_summary["total_transaction_amount"] = sum(transaction_amounts)
    )
    .reset_index()

)

print("\nReturn Summary:")
display(return_summary)

plt.figure(figsize=(6,3))

sns.barplot(
    data=return_summary,
    x="returns",
    y="Transactions"
)

plt.title("Returned vs Non-Returned Transactions")
plt.xlabel("Return Status")
plt.ylabel("Number of Transactions")
plt.tight_layout()
plt.show()
```

```
# =====  
# SECTION N – CORRELATION HEATMAP  
# =====
```

```
# -----  
# STEP 31 – CUSTOMER METRIC CORRELATION  
# -----
```

```
correlation_columns = [  
    "recency",  
    "frequency",  
    "monetary",  
    "R_Score",  
    "F_Score",  
    "M_Score",  
    "RFM_Score",  
    "churn"  
]  
  
available_correlation_columns = [  
    col for col in correlation_columns  
    if col in rfm.columns  
]  
  
if len(available_correlation_columns) >= 2:  
  
    correlation_matrix = rfm[  
        available_correlation_columns
```

```

].corr()

plt.figure(figsize=(6,3))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0
)

plt.title("RFM and Churn Correlation Heatmap")
plt.tight_layout()
plt.show()

```

```

# =====
# SECTION 0 – TOP CUSTOMERS
# =====

# -----
# STEP 32 – TOP 10 CUSTOMERS BY REVENUE
# -----

if "monetary" in rfm.columns:

```

```
top_customers = (
    rfm
    .sort_values(
        "monetary",
        ascending=False
    )
    .head(10)
    .copy()
)

display(top_customers)

plt.figure(figsize=(6,3))

sns.barplot(
    data=top_customers,
    x="customer_id",
    y="monetary"
)

plt.title("Top 10 Customers by Monetary Value")
plt.xlabel("Customer ID")
plt.ylabel("Total Spend")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```

# =====
# SECTION P – HIGH-VALUE AT-RISK CUSTOMERS
# =====

# -----
# STEP 33 – IDENTIFY HIGH-VALUE AT-RISK CUSTOMERS
# -----

if {
    "R_Score",
    "M_Score",
    "churn"
}.issubset(rfm.columns):

    high_value_at_risk = rfm[
        (
            (rfm["R_Score"] <= 2)
            &
            (rfm["M_Score"] >= 4)
            &
            (rfm["churn"] == 0)
        )
    ].copy()

    high_value_at_risk = high_value_at_risk.sort_values(
        "monetary",
        ascending=False
    )

```

```
print(
    "\nHigh-Value At-Risk Customers:",
    len(high_value_at_risk)
)

display(
    high_value_at_risk.head(20)
)
```

```
# =====
# SECTION Q – DASHBOARD-READY KPI TABLE
# =====

# -----
# STEP 34 – CREATE KPI TABLE
# -----
```

```
dashboard_kpis = pd.DataFrame({
    "KPI": [
        "Total Customers",
        "Total Transactions",
        "Total Revenue",
        "Average Order Value",
        "Total Quantity Sold",
        "Total Returns",
        "Churn Rate (%)"
    ]
})
```

```
    ],
    "Value": [
        total_customers,
        total_transactions,
        round(total_revenue, 2),
        round(average_order_value, 2),
        total_quantity,
        total_returns,
        round(churn_rate, 2)
    ]
})
```

```
print("\n===== DASHBOARD KPI TABLE =====\n")
```

```
display(dashboard_kpis)
```

```
# =====
# SECTION R – EXPORT VISUALIZATION DATA
# =====

# -----
# STEP 35 – CREATE OUTPUT FOLDER
# -----
```

```
output_folder = Path("phase6_visualization_outputs")
```

```
output_folder.mkdir(  
    exist_ok=True  
)  
  
# -----  
# STEP 36 - SAVE KPI DATA  
# -----  
  
dashboard_kpis.to_csv(  
    output_folder / "dashboard_kpis.csv",  
    index=False  
)  
  
# -----  
# STEP 37 - SAVE MONTHLY SALES  
# -----  
  
monthly_sales.to_csv(  
    output_folder / "monthly_sales_visualization.csv",  
    index=False  
)
```

```
# -----  
# STEP 38 - SAVE YEARLY SALES  
# -----  
  
yearly_sales.to_csv(  
    output_folder / "yearly_sales_visualization.csv",  
    index=False  
)  
  
# -----  
# STEP 39 - SAVE CATEGORY SUMMARY  
# -----  
  
category_summary.to_csv(  
    output_folder / "category_visualization.csv",  
    index=False  
)  
  
# -----  
# STEP 40 - SAVE PAYMENT SUMMARY  
# -----  
  
payment_summary.to_csv(  
    output_folder / "payment_method_visualization.csv",  
    index=False  
)
```

```
# STEP 41 - SAVE DAY SUMMARY
# -----

day_summary.to_csv(
    output_folder / "day_of_week_visualization.csv",
    index=False
)

# -----
# STEP 42 - SAVE HOURLY SUMMARY
# -----

hour_summary.to_csv(
    output_folder / "hourly_purchase_visualization.csv",
    index=False
)

# -----
# STEP 43 - SAVE QUANTITY SUMMARY
# -----

quantity_summary.to_csv(
    output_folder / "quantity_visualization.csv",
    index=False
)
```

```
# -----  
# STEP 44 - SAVE SEGMENT SUMMARY  
# -----
```

```
if not segment_summary.empty:  
  
    segment_summary.to_csv(  
        output_folder / "customer_segment_visualization.csv",  
        index=False  
    )
```

```
# -----  
# STEP 45 - SAVE CHURN BY SEGMENT  
# -----
```

```
if "segment_churn" in globals():  
  
    segment_churn.to_csv(  
        output_folder / "segment_churn_visualization.csv",  
        index=False  
    )
```

```
# -----  
# STEP 46 - SAVE AGE SUMMARY  
# -----
```

```
if "age_summary" in globals():  
  
    age_summary.to_csv(  
        output_folder / "age_group_visualization.csv",  
        index=False  
    )
```

```
# -----  
# STEP 47 - SAVE GENDER SUMMARY  
# -----
```

```
if "gender_summary" in globals():  
  
    gender_summary.to_csv(  
        output_folder / "gender_visualization.csv",  
        index=False  
    )
```

```
# =====  
# SECTION 5 – CREATE MASTER VISUALIZATION DATASET  
# =====  
  
# -----  
# STEP 48 – MASTER SUMMARY  
# -----
```

```
master_visualization_summary = pd.DataFrame({  
    "Metric": [  
        "Total Customers",  
        "Total Transactions",  
        "Total Revenue",  
        "Average Order Value",  
        "Total Quantity",  
        "Average Quantity",  
        "Total Returns",  
        "Churn Rate"  
    ],  
    "Value": [  
        total_customers,  
        total_transactions,  
        total_revenue,  
        average_order_value,  
        total_quantity,  
        average_quantity,
```

```

        total_returns,
        churn_rate
    ]
})

master_visualization_summary.to_csv(
    output_folder / "master_visualization_summary.csv",
    index=False
)

# =====
# SECTION T - FINAL OUTPUT CHECK
# =====

# -----
# STEP 49 - LIST GENERATED FILES
# -----

print("\n=====")
print("STEP 6 COMPLETED SUCCESSFULLY")
print("=====")

print("\nVisualization Output Files:\n")

for file in sorted(output_folder.iterdir()):
    print("✓", file.name)

```

```

# -----
# STEP 50 - FINAL DASHBOARD KPI DISPLAY
# -----

print("\n===== FINAL DASHBOARD KPIs =====\n")

print(f"Total Customers      : {total_customers:,}")
print(f"Total Transactions     : {total_transactions:,}")
print(f"Total Revenue          : {total_revenue:,.2f}")
print(f"Average Order Value    : {average_order_value:,.2f}")
print(f"Total Quantity Sold    : {total_quantity:,}")
print(f"Total Returns          : {total_returns:,}")
print(f"Churn Rate             : {churn_rate:.2f}%")

print("\nStep 6 visualization and dashboard preparation completed.")

```

```

Transactions Shape: (250000, 17)
Customer-Level Shape: (49673, 20)
RFM Shape: (49673, 12)

```

Transaction Columns:

```

['customer_id', 'purchase_date', 'product_category', 'product_price', 'quantity', 'total_purchase_amount', 'payment_method', 'customer_age', 'returns',
'customer_name', 'gender', 'churn', 'year', 'month', 'month_name', 'quarter', 'day_of_week']

```

Customer Columns:

```

['customer_id', 'customer_name', 'total_spend', 'purchase_frequency', 'total_quantity', 'average_order_value', 'average_product_price', 'first_purchase_date', 'last_purchase_date', 'total_returns', 'churn', 'recency', 'customer_tenure_days', 'customer_tenure_months', 'return_rate', 'return_rate_pct', 'customer_age', 'gender', 'preferred_category', 'preferred_payment_method']

```

RFM Columns:

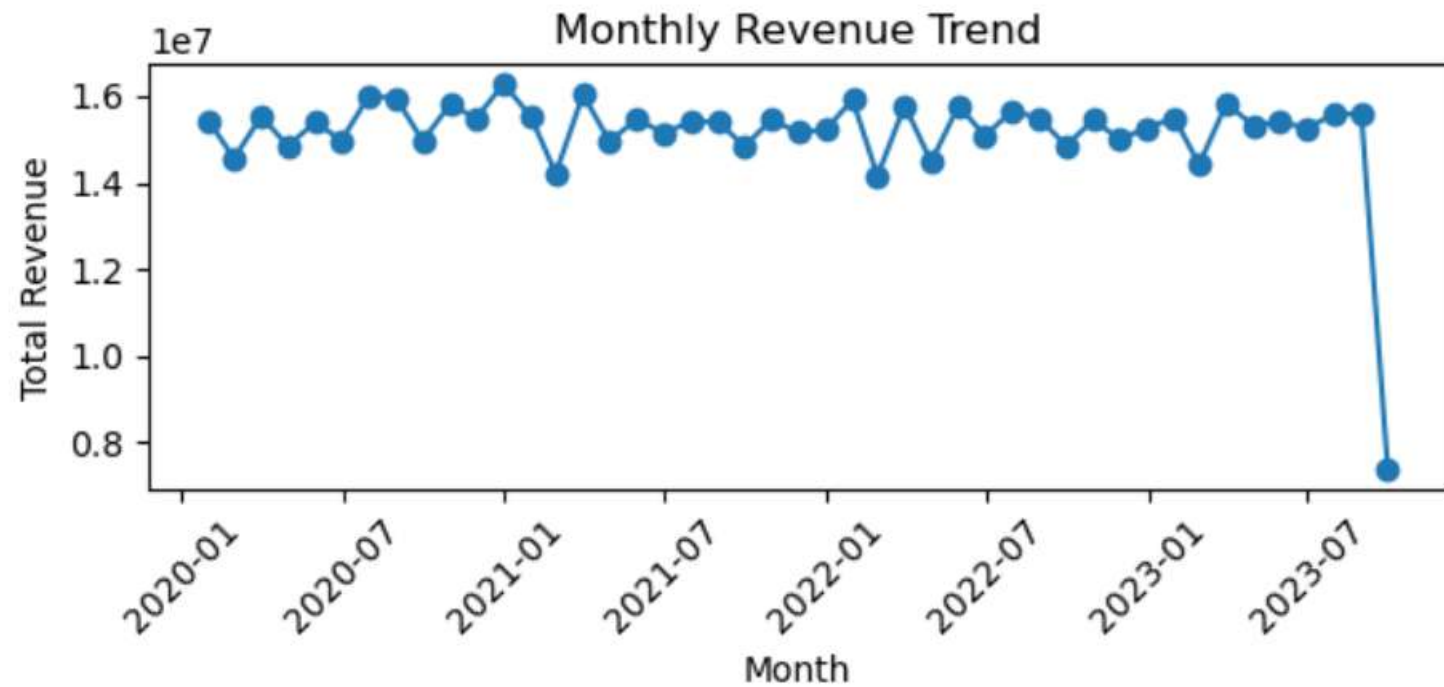
```
['customer_id', 'customer_name', 'recency', 'frequency', 'monetary', 'churn', 'R_Score', 'F_Score', 'M_Score', 'RFM_Score', 'RFM_Score_Code', 'Customer_Segment']
```

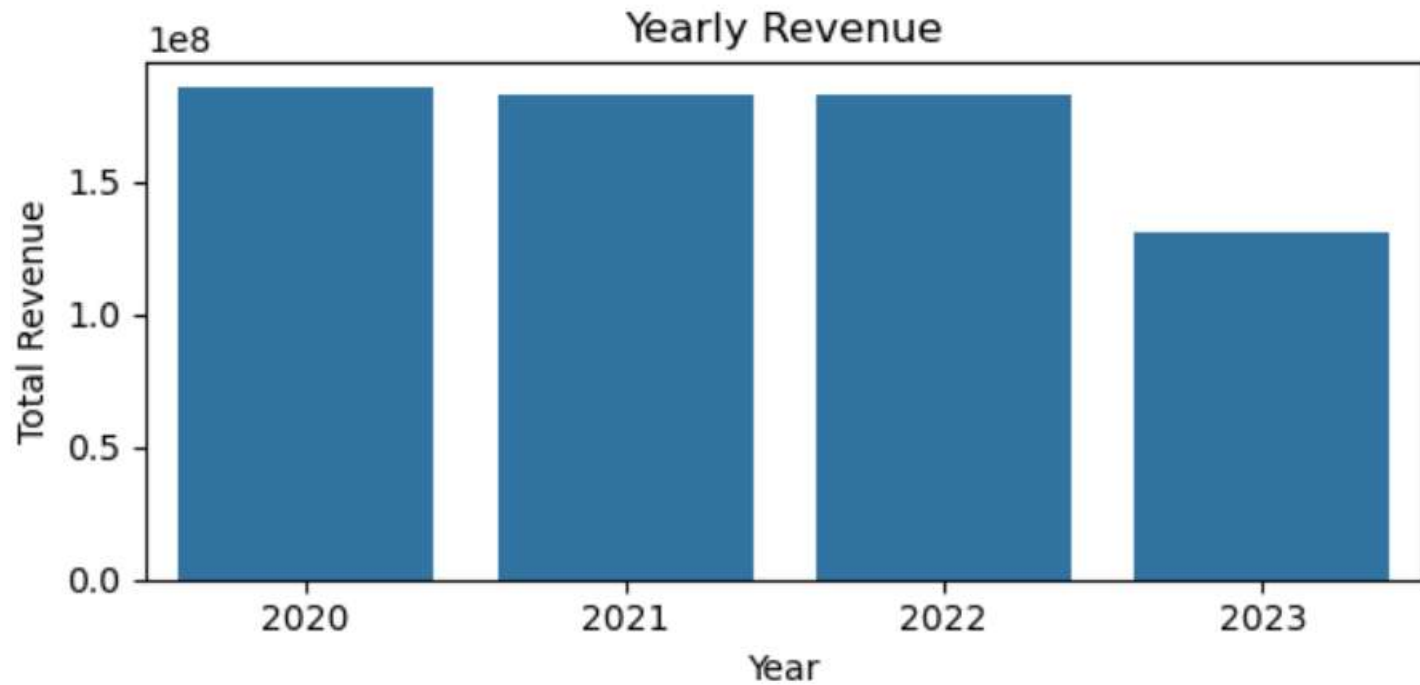
===== BUSINESS KPIs =====

```
Total Transactions      : 250000
Total Customers         : 49673
Total Revenue           : 681342683
Average Order Value     : 2725.37
Total Quantity Sold     : 749724
Average Quantity/Order  : 3.0
Total Returns           : 100769.0
Overall Churn Rate (%)  : 19.95
```

C:\Users\Ankit\AppData\Local\Temp\ipykernel_25920\153436312.py:112: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.

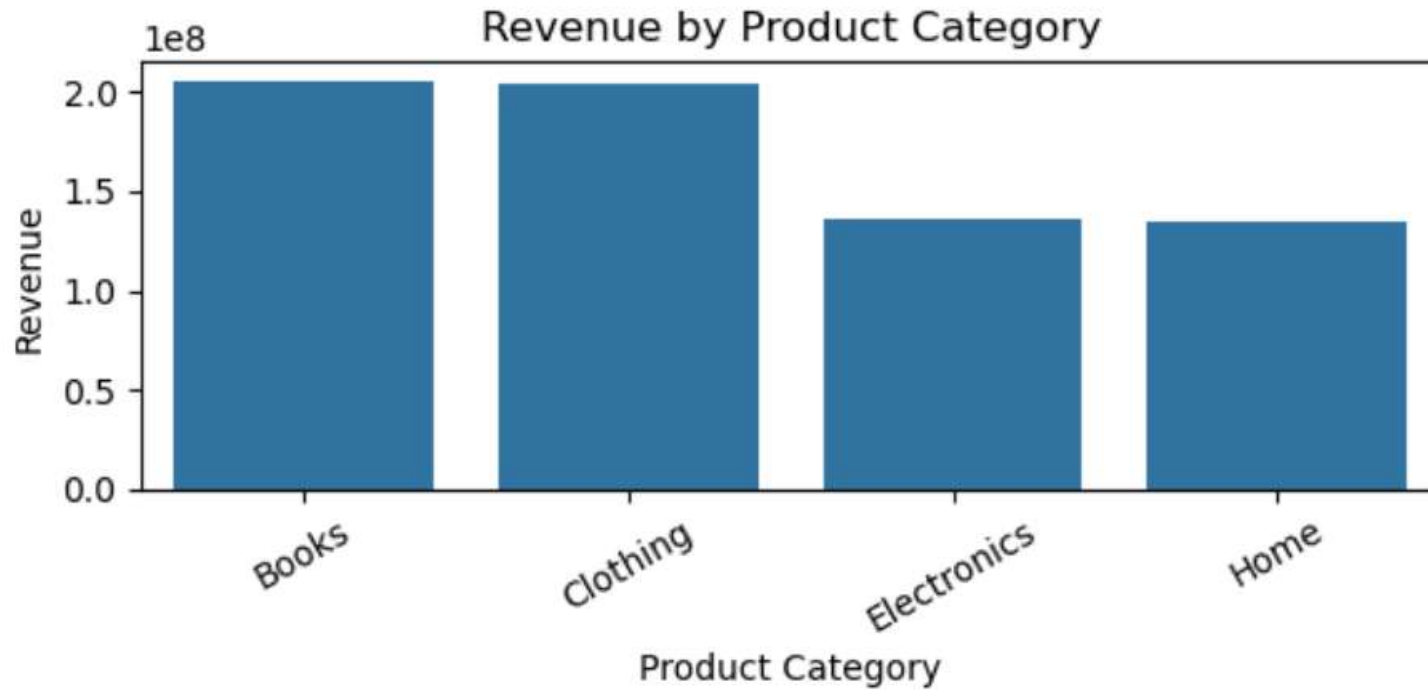
```
.resample("M")["total_purchase_amount"]
```



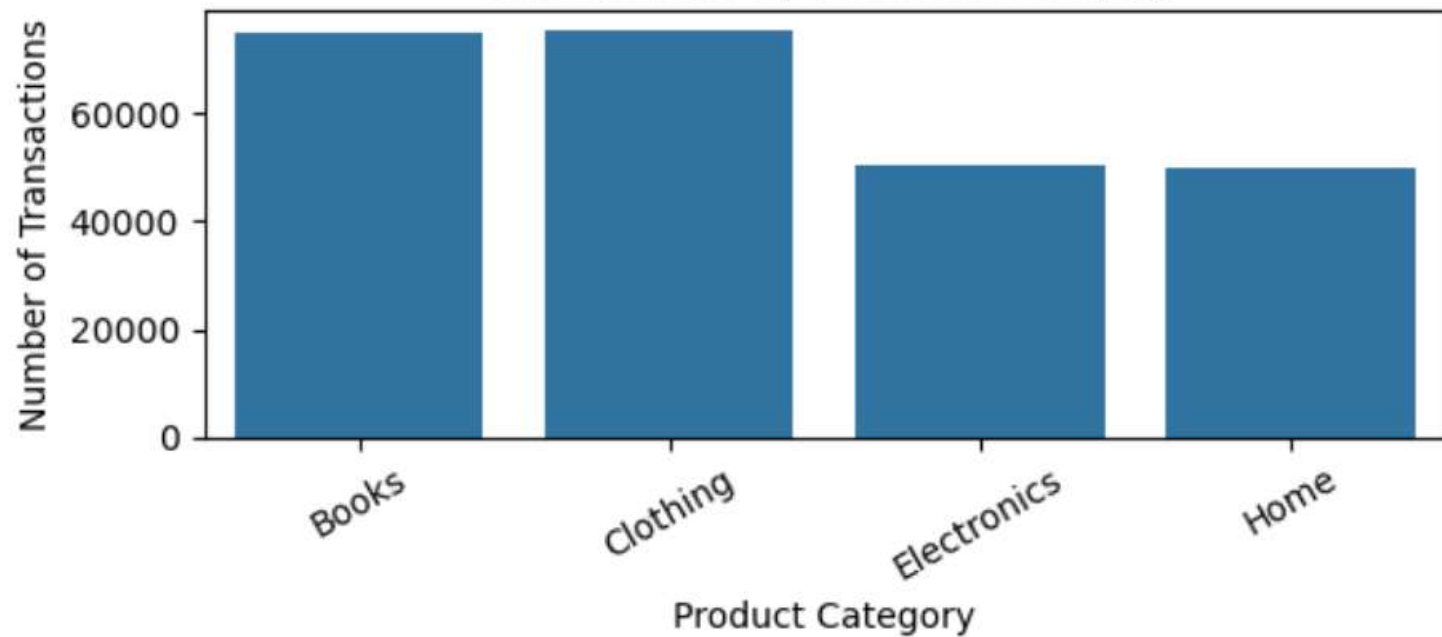


Category Performance:

	product_category	Revenue	Transactions	Quantity	Average_Order_Value
0	Books	204939601	74912	223876	2735.737946
1	Clothing	204532405	75052	225322	2725.209255
2	Electronics	136599467	50185	150828	2721.918243
3	Home	135271210	49851	149698	2713.510461

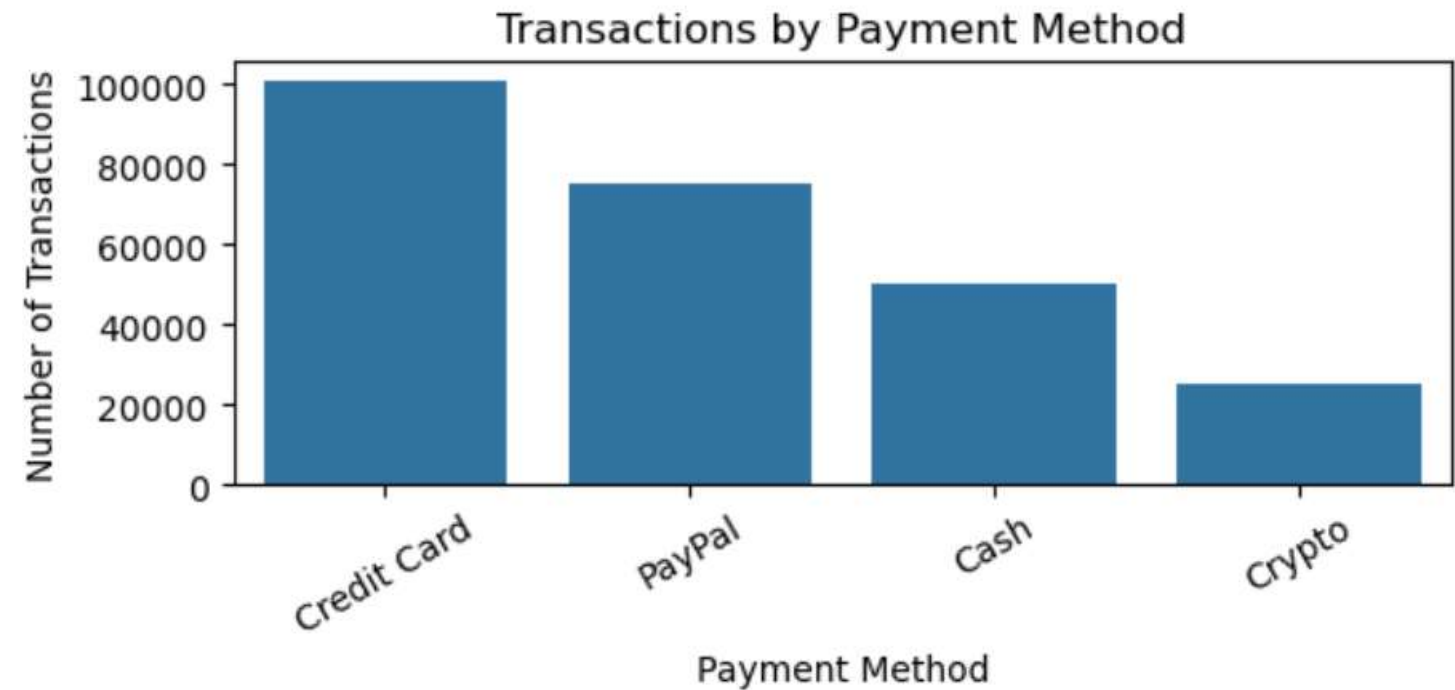


Transactions by Product Category



Payment Method Performance:

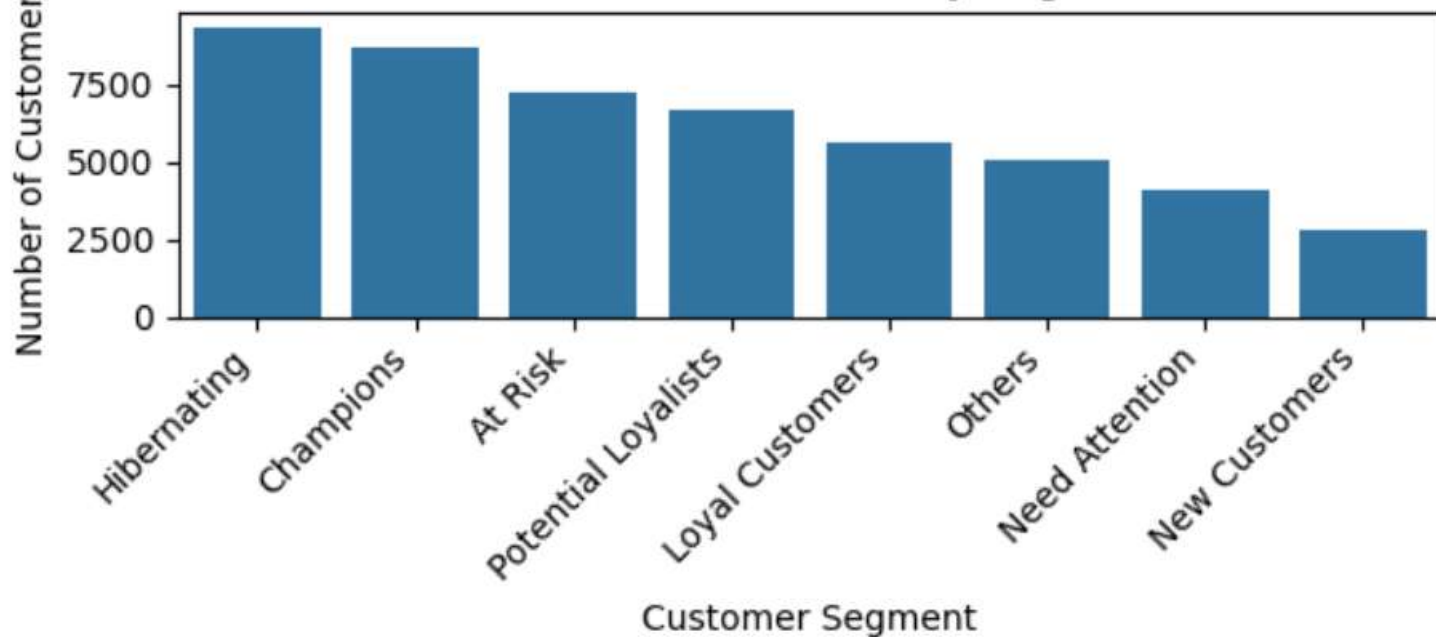
	payment_method	Transactions	Revenue	Average_Order_Value
1	Credit Card	100486	274152396	2728.264594
3	PayPal	74837	204026613	2726.279955
0	Cash	49894	135705551	2719.877160
2	Crypto	24783	67458123	2721.951459



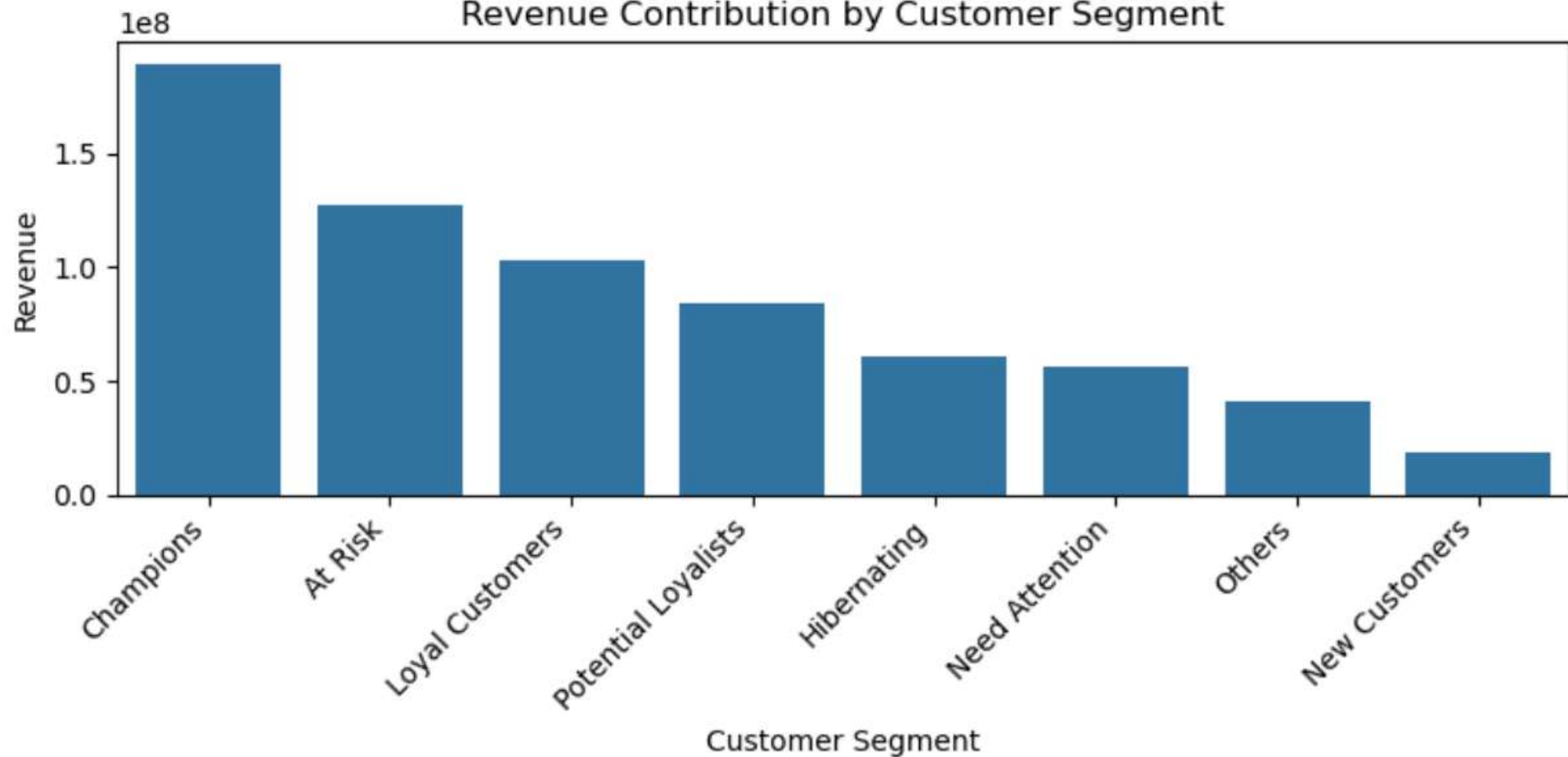
Customer Segment Summary:

	Customer_Segment	Customers	Revenue	Average_Spend	Average_Frequency	Average_Recency
1	Champions	8692	189393382	21789.390474	7.648642	59.891855
0	At Risk	7271	127579032	17546.284142	6.159813	404.240270
3	Loyal Customers	5644	103289001	18300.673459	6.973600	153.387845
7	Potential Loyalists	6722	84076914	12507.722999	4.534662	64.596102
2	Hibernating	9347	60665965	6490.420991	2.590350	578.922007
4	Need Attention	4074	56634408	13901.425626	4.189985	314.092293
6	Others	5111	41409471	8102.029153	3.912150	254.117785
5	New Customers	2812	18294510	6505.871266	2.706970	66.412162

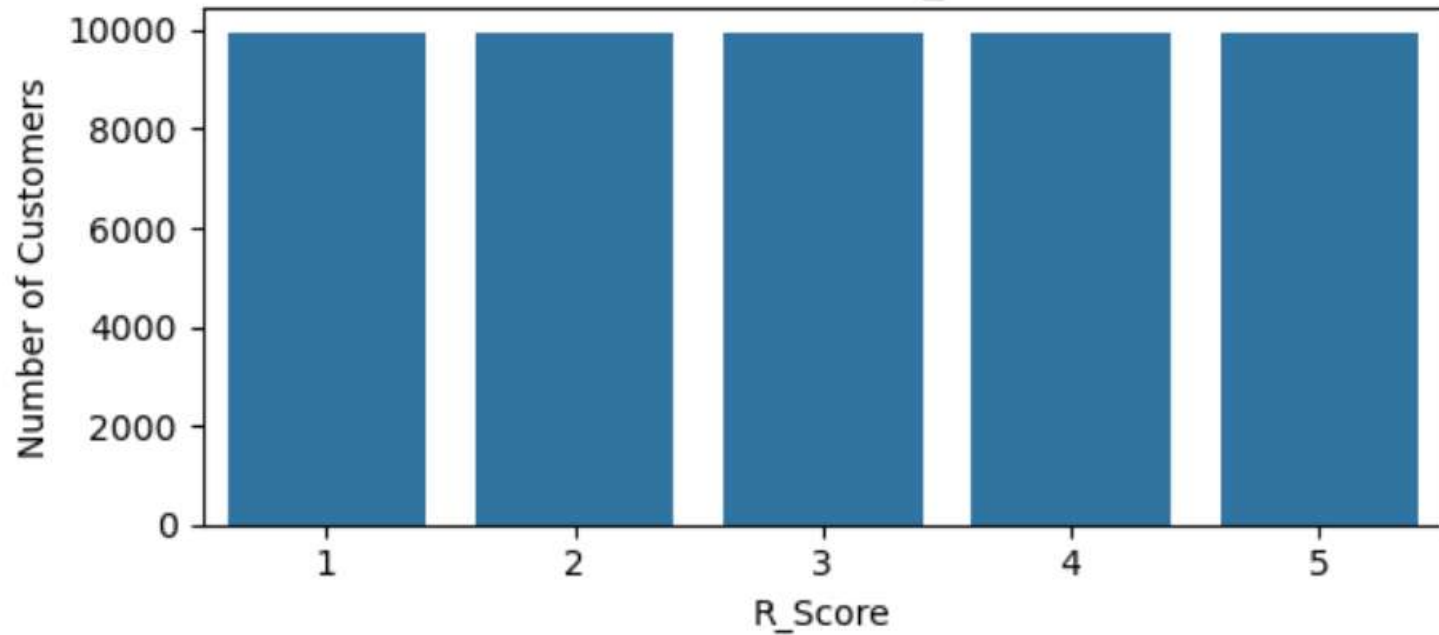
Customer Distribution by Segment



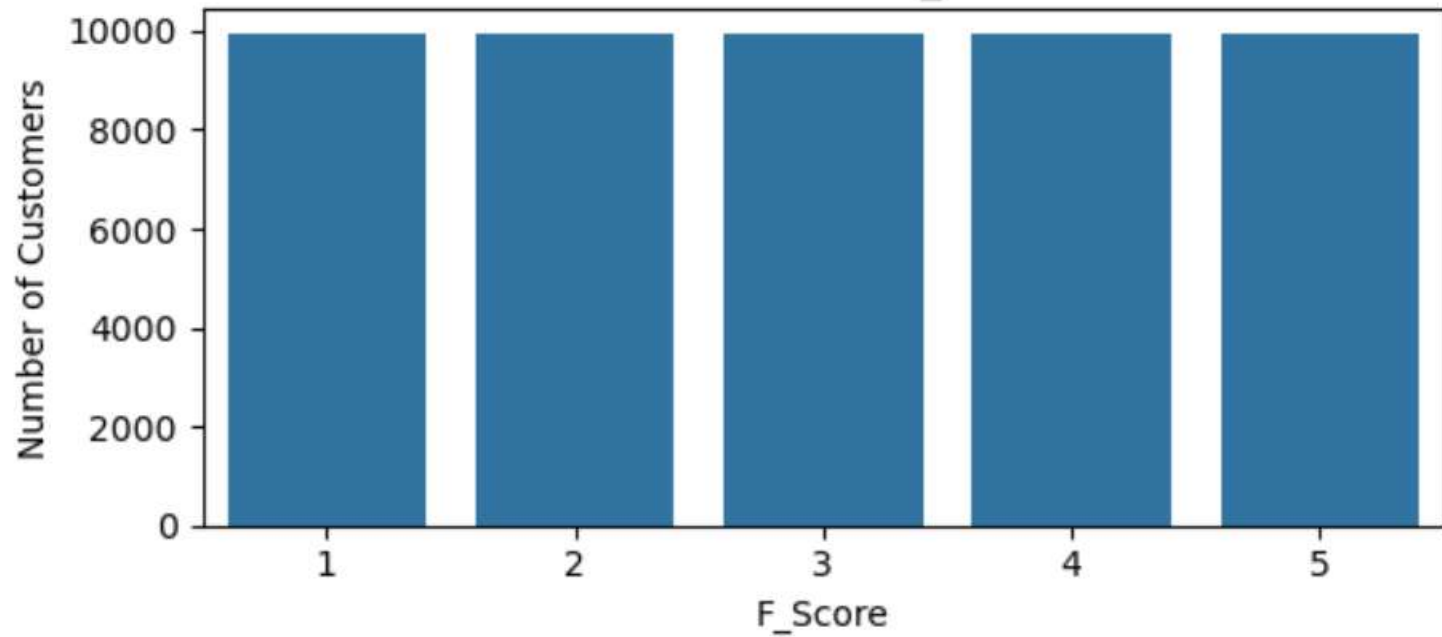
Revenue Contribution by Customer Segment



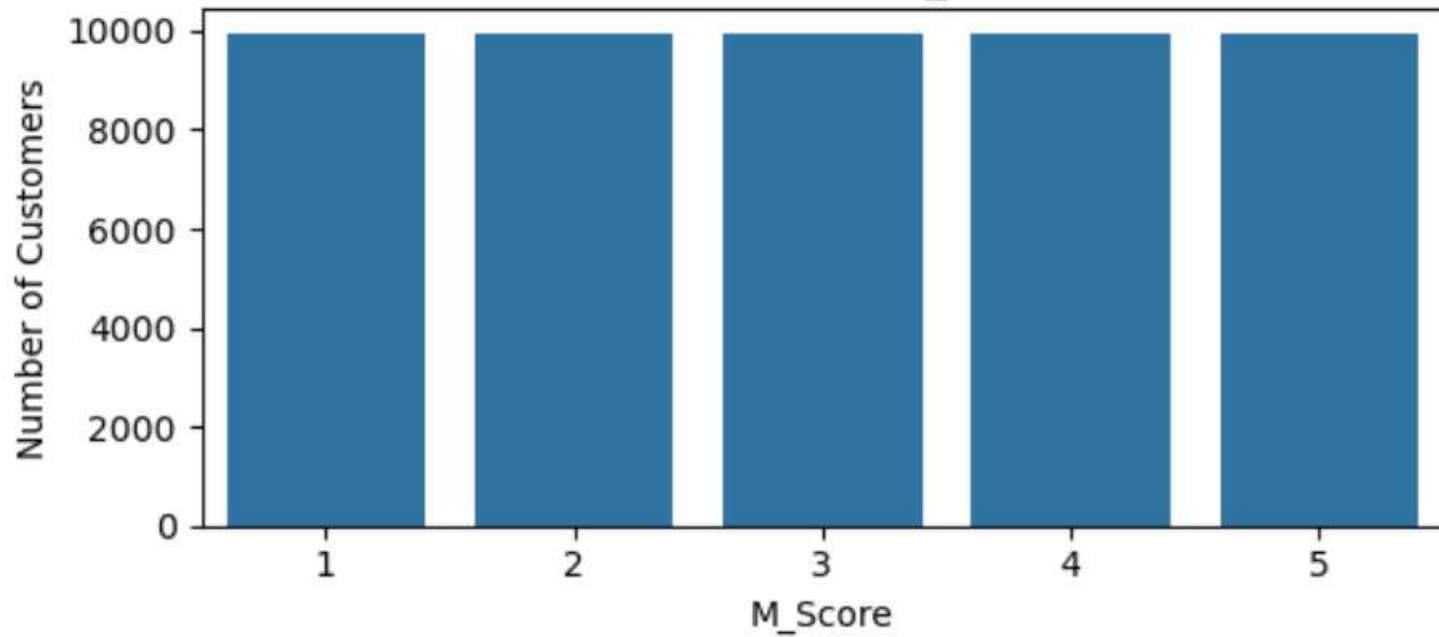
Distribution of R_Score



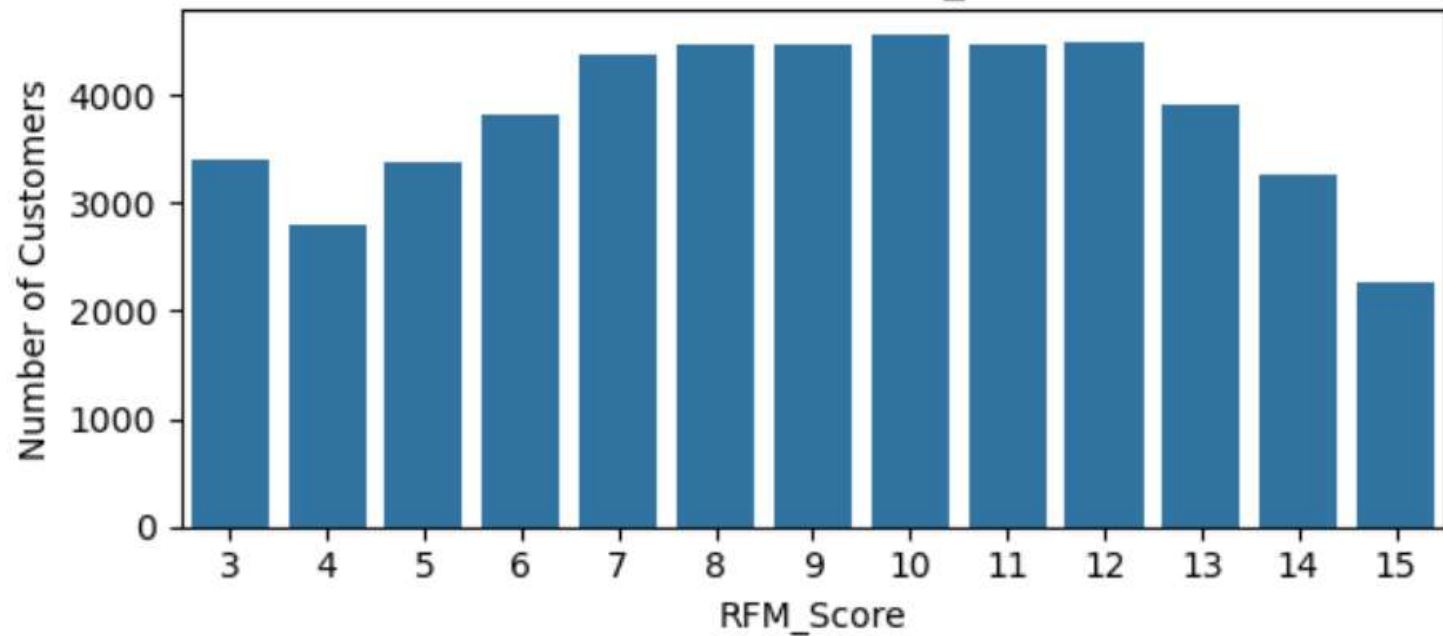
Distribution of F_Score



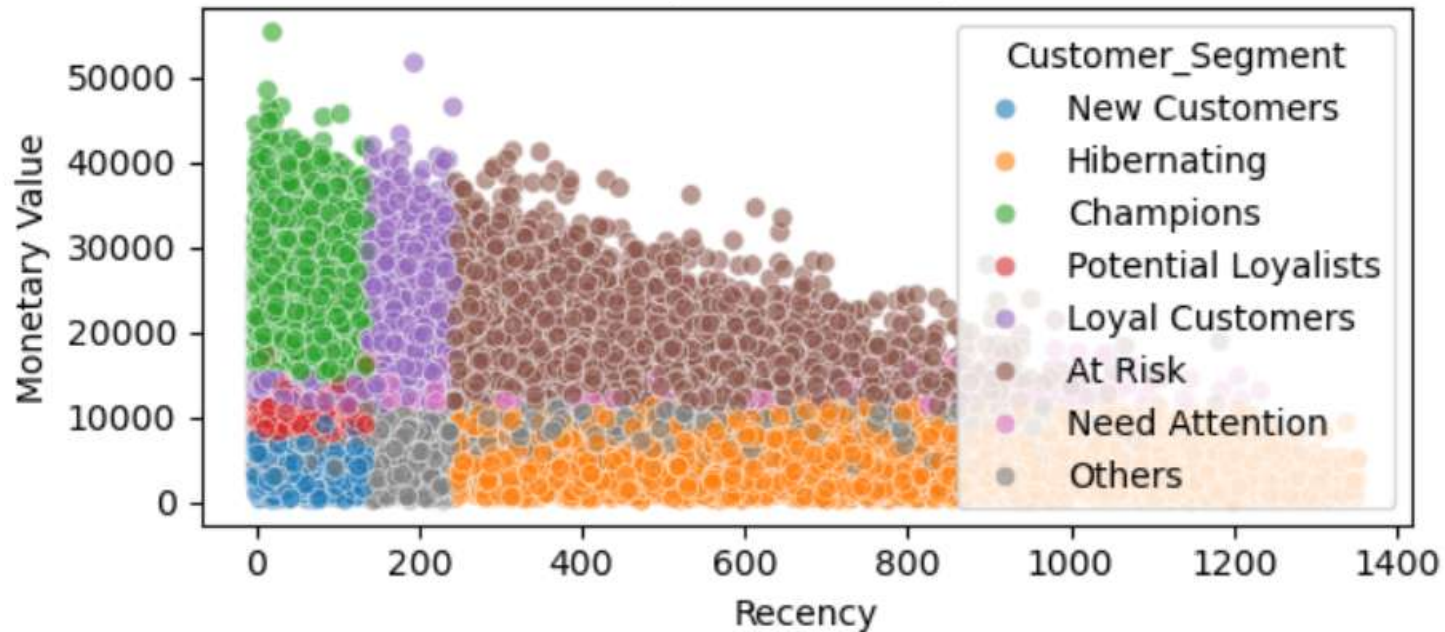
Distribution of M_Score



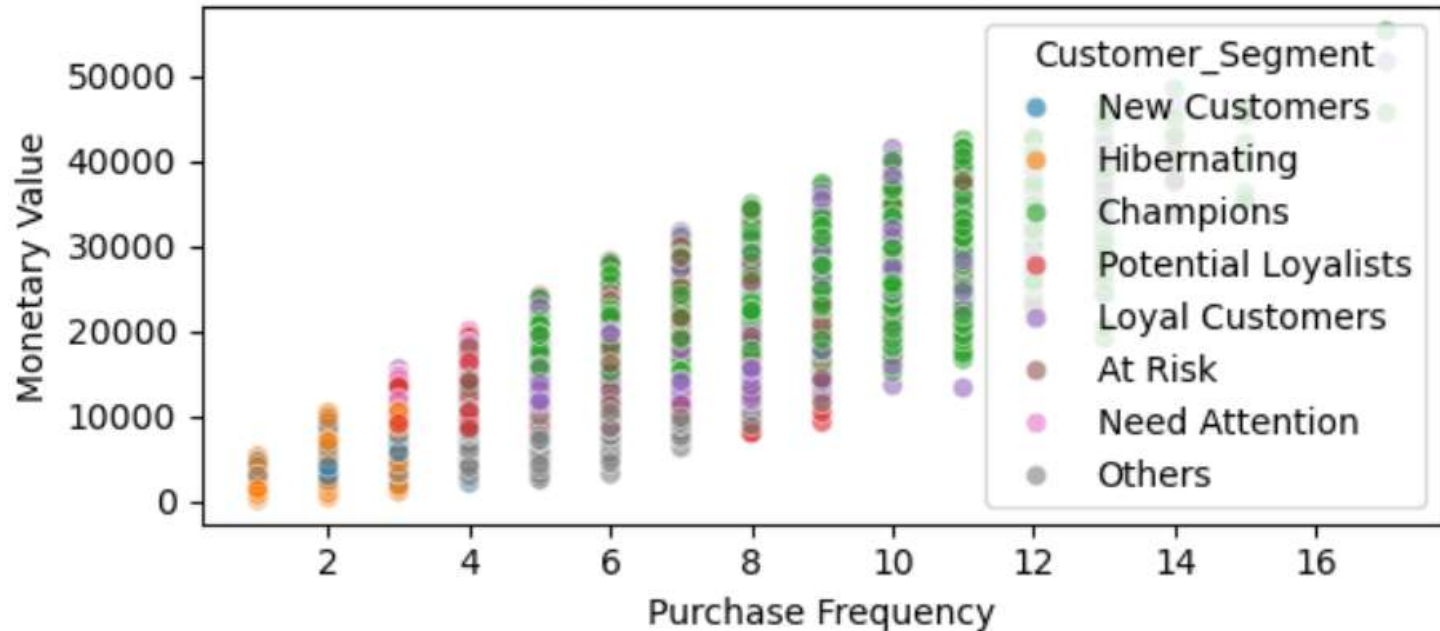
Distribution of RFM_Score



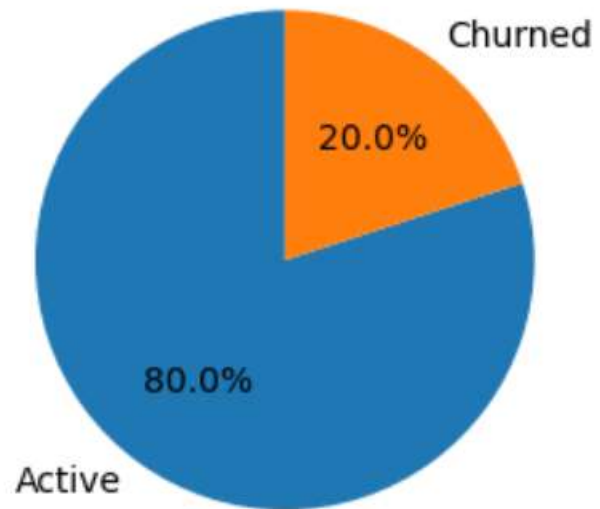
Recency vs Monetary Value



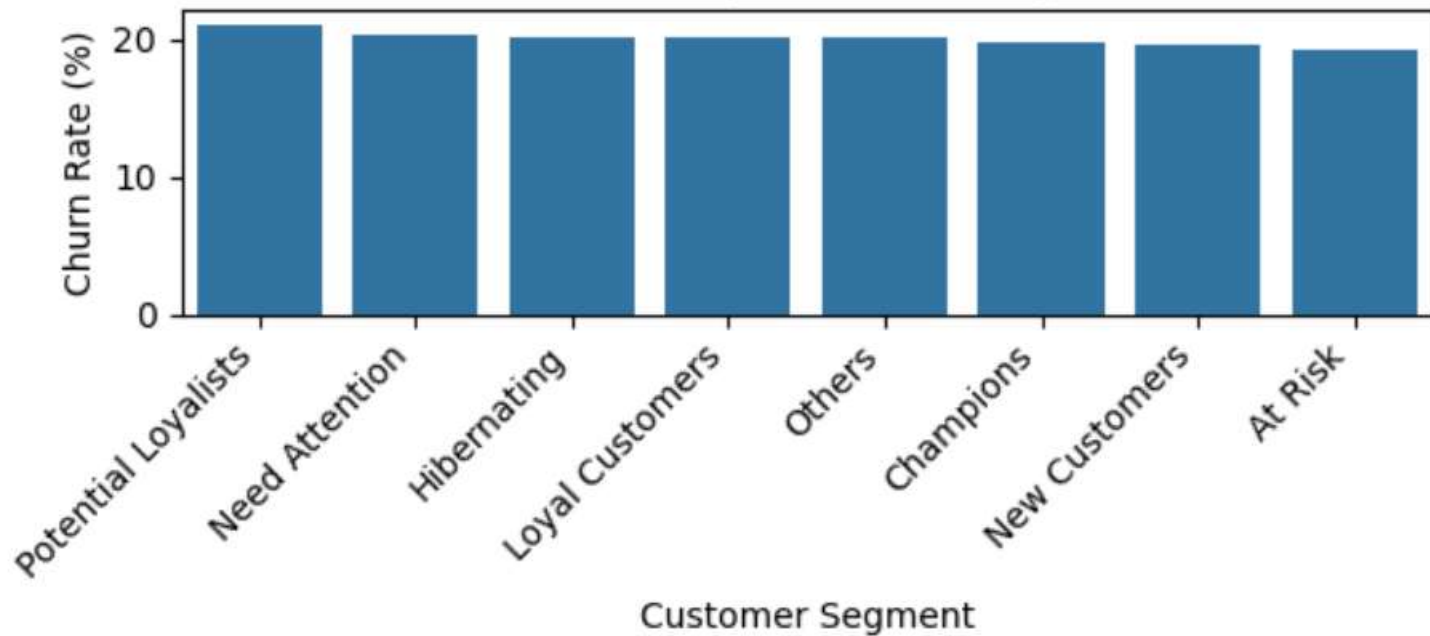
Frequency vs Monetary Value



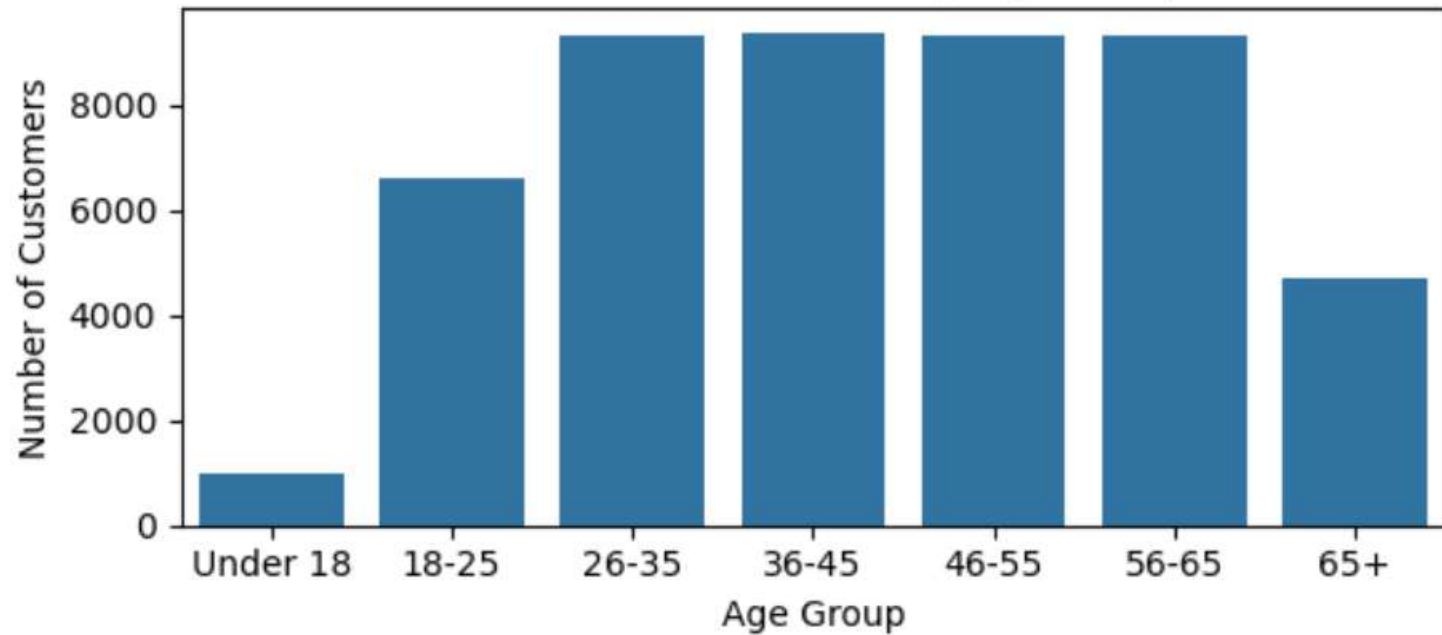
Customer Churn Distribution



Churn Rate by Customer Segment

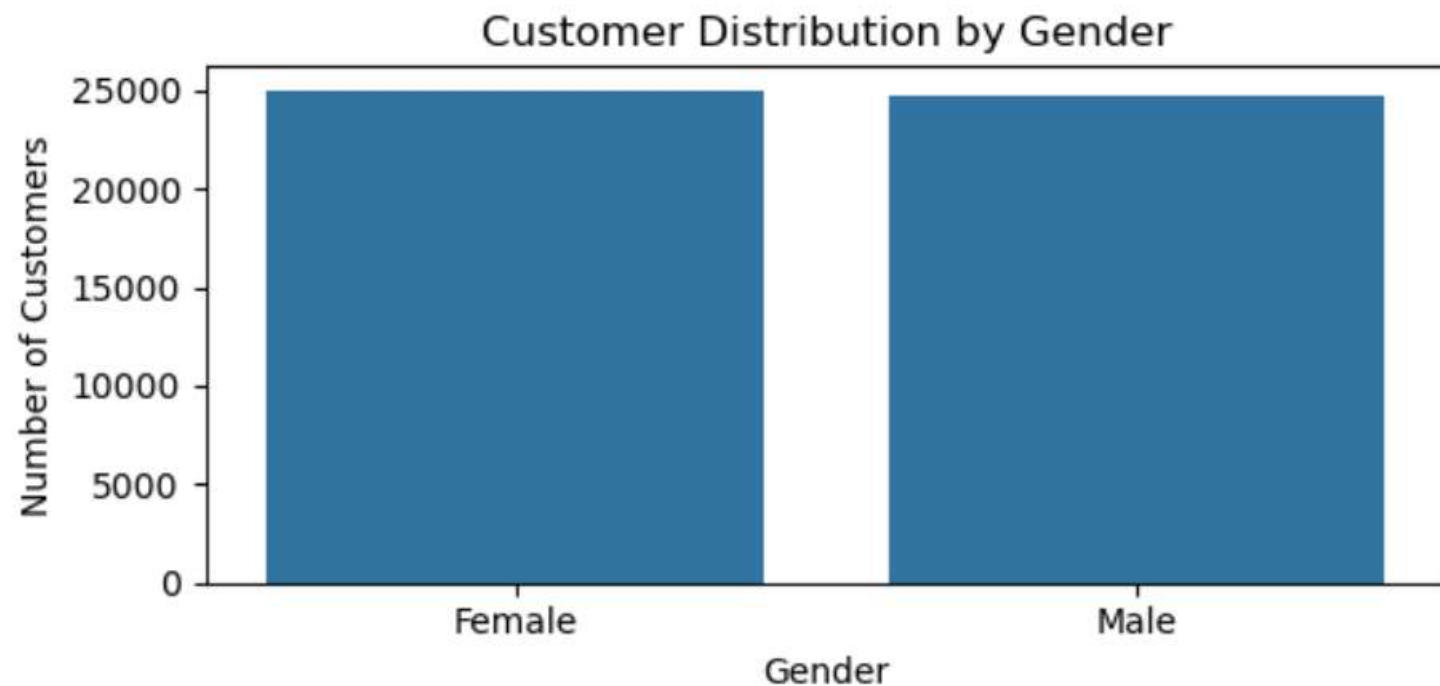


Customer Distribution by Age Group

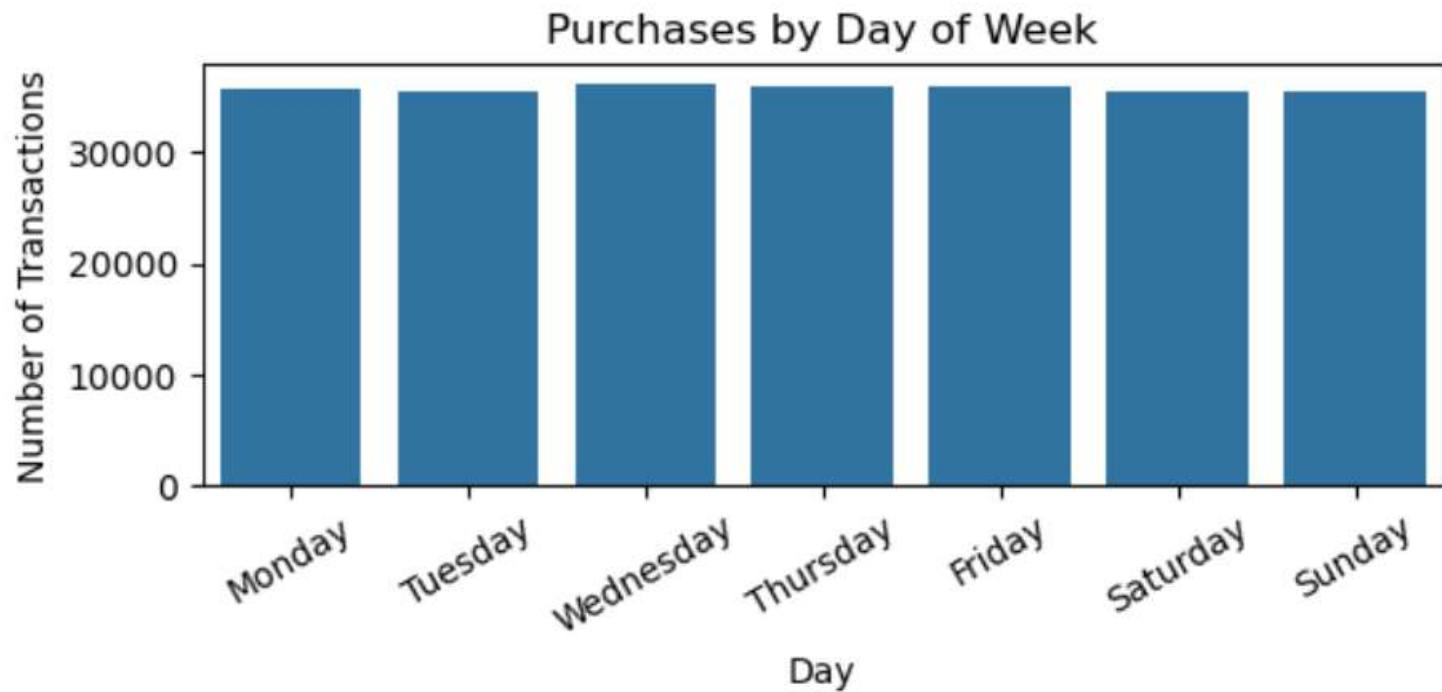


Gender Summary:

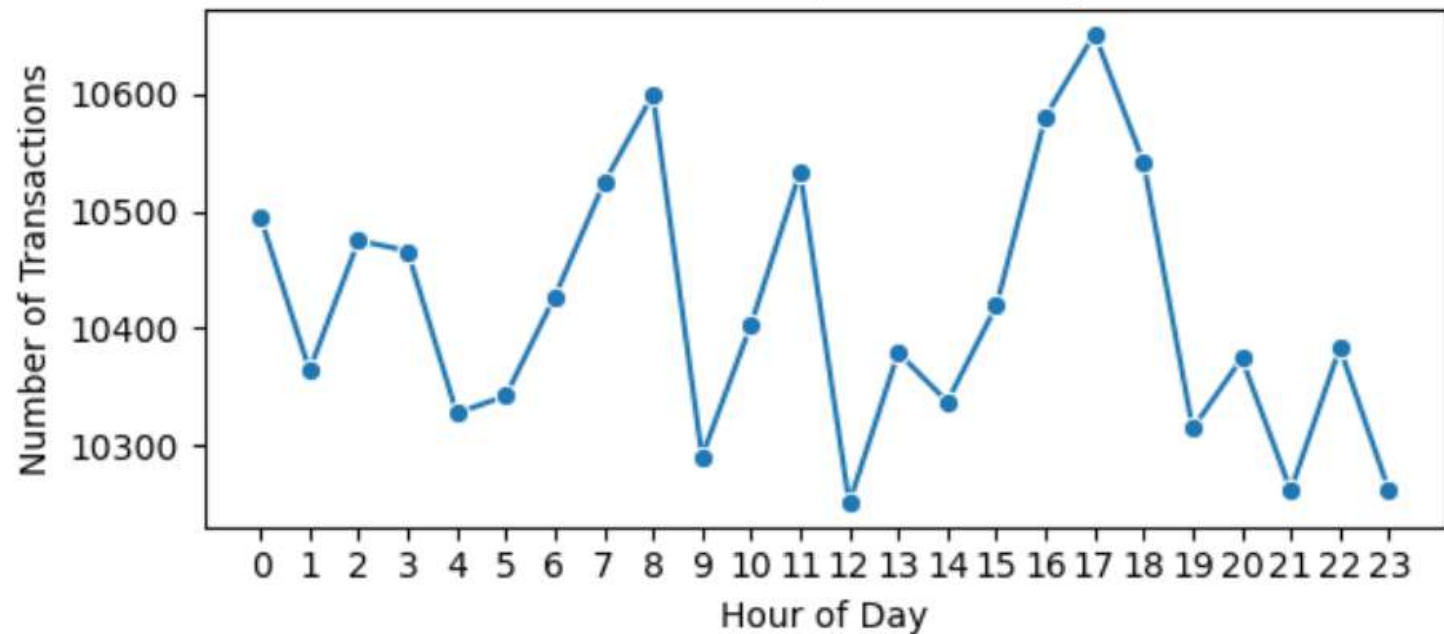
	gender	Customers	Revenue
0	Female	24958	342462421
1	Male	24715	338880262



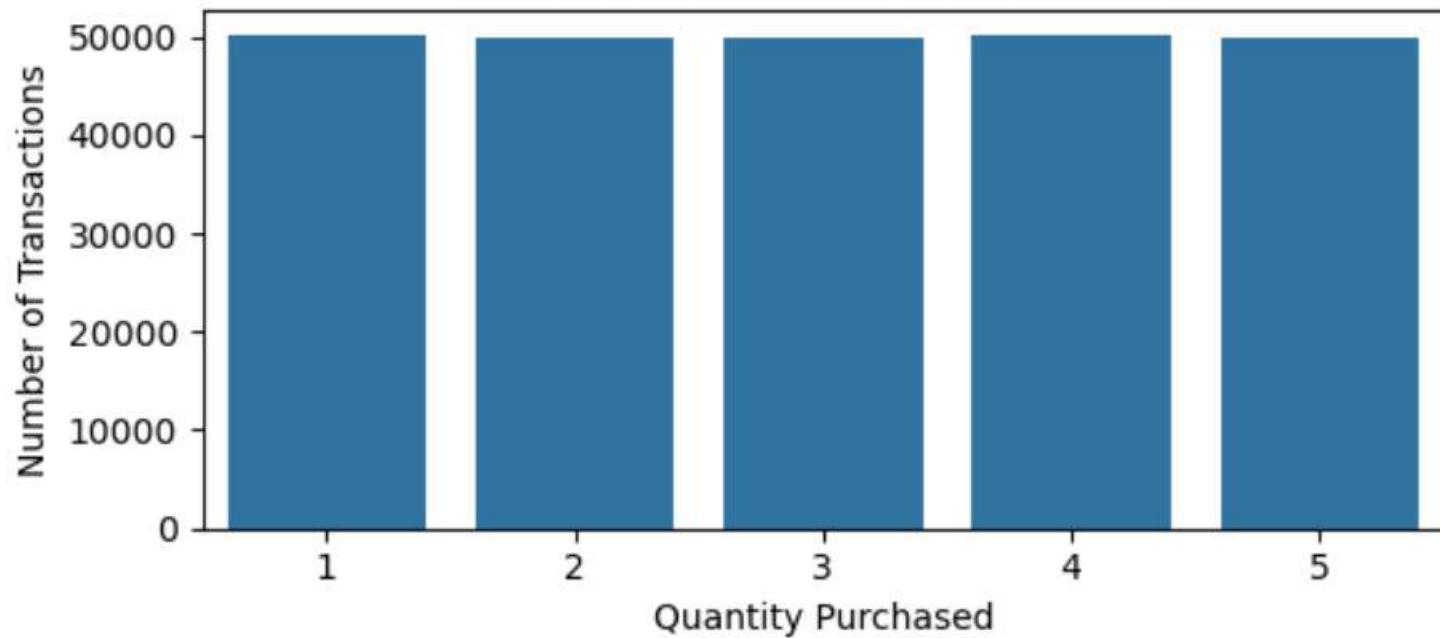
Gender



Purchases by Hour of Day

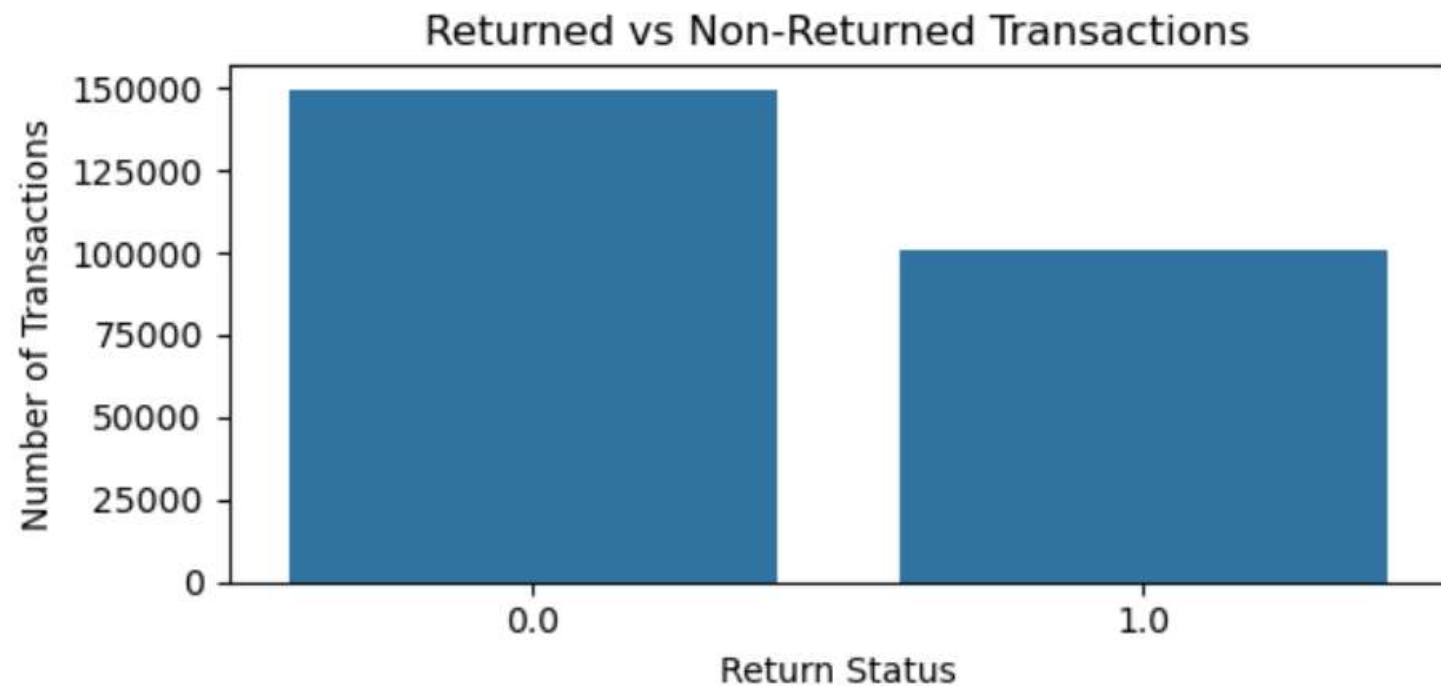


Transaction Distribution by Quantity

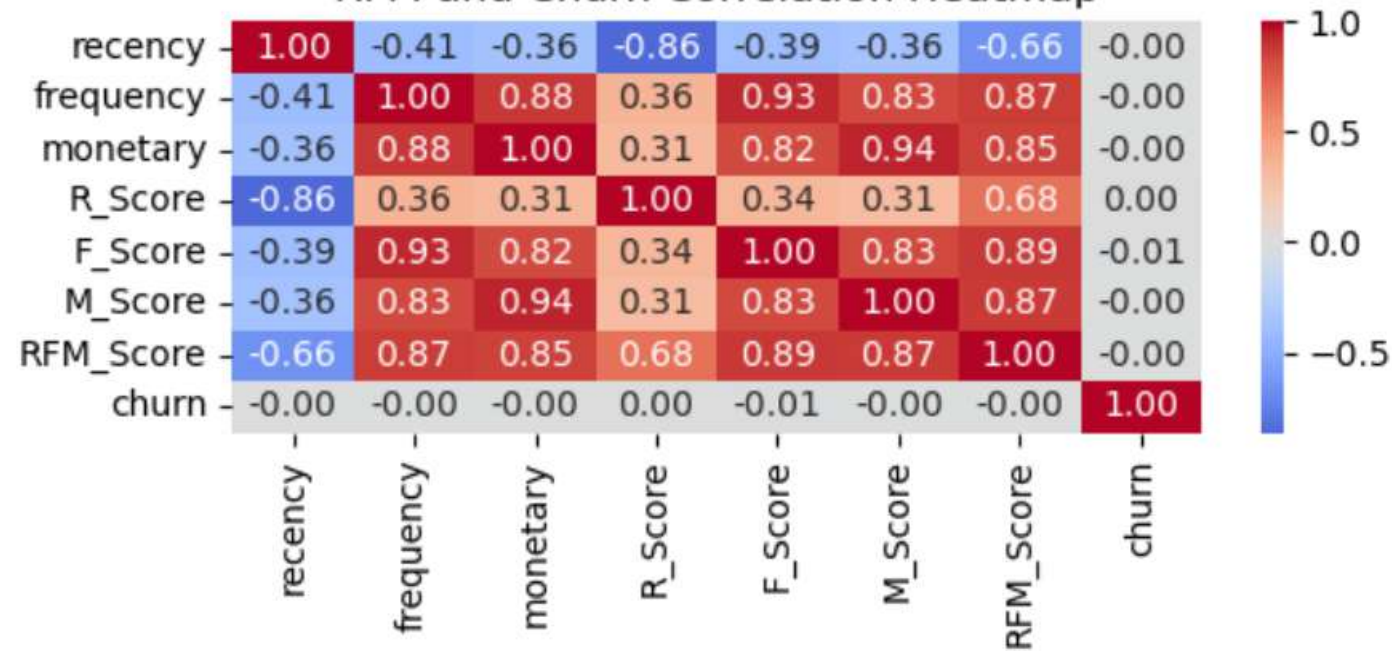


Return Summary:

	returns	Transactions	Revenue
0	0.0	149231	406676297
1	1.0	100769	274666386

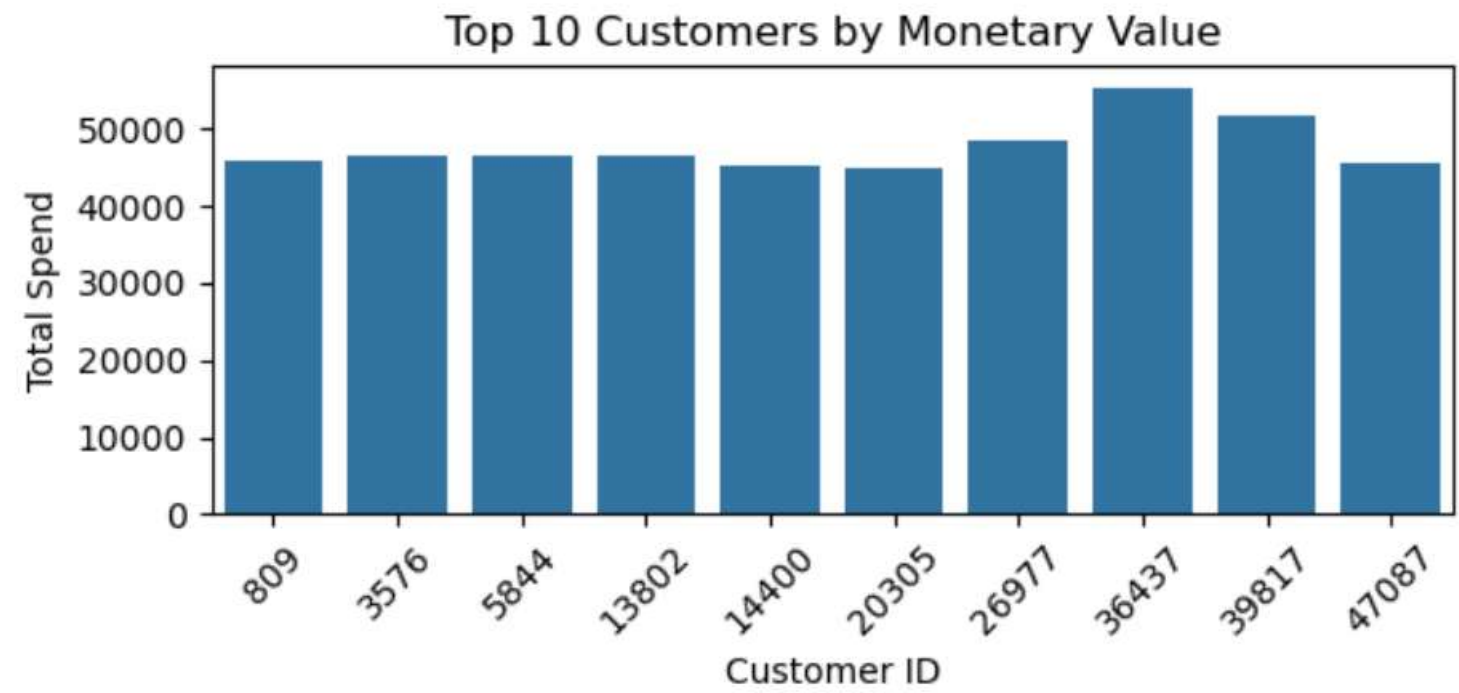


RFM and Churn Correlation Heatmap



	customer_id	customer_name	recency	frequency	monetary	churn	R_Score	F_Score	M_Score	RFM_Score	RFM_Score_Code	Customer_Segment
36211	36437	Jesse Gray	20	17	55339	0	5	5	5	15	555	Champions
39561	39817	Dr. Courtney Foster	194	17	51718	0	3	5	5	13	355	Loyal Customers
26818	26977	Felicia Escobar	14	14	48497	0	5	5	5	15	555	Champions
5809	5844	Maria Owens	31	12	46541	0	5	5	5	15	555	Champions
3555	3576	Aaron Harrell	242	14	46518	0	3	5	5	13	355	Loyal Customers

13718	13802	Jonathan Schroeder	16	13	46464	0	5	5	5	15	555	Champions
802	809	Taylor Bishop	20	15	45738	0	5	5	5	15	555	Champions
46785	47087	Debbie Lopez	104	17	45700	1	4	5	5	14	455	Champions
14313	14400	Patricia Petersen	83	15	45334	0	4	5	5	14	455	Champions
20186	20305	Randall Hobbs	22	14	45044	0	5	5	5	15	555	Champions



High-Value At-Risk Customers: 4226

	customer_id	customer_name	recency	frequency	monetary	churn	R_Score	F_Score	M_Score	RFM_Score	RFM_Score_Code	Customer_Segment
27976	28144	Lindsay Morse	315	14	41347	0	2	5	5	12	255	At Risk
48150	48471	David Duran	308	14	40436	0	2	5	5	12	255	At Risk
6742	6789	Andrew Salazar	294	11	39574	0	2	5	5	12	255	At Risk
31467	31659	Thomas Rosales	368	10	39068	0	2	5	5	12	255	At Risk
24833	24981	Samuel Lopez PhD	307	11	38511	0	2	5	5	12	255	At Risk
28918	29096	Amy Mitchell	430	13	37978	0	1	5	5	11	155	At Risk
29108	29286	Deborah Schwartz	282	11	37958	0	2	5	5	12	255	At Risk
28920	29098	Erin Romero	245	11	37770	0	2	5	5	12	255	At Risk
45686	45979	Kelly Smith	340	14	37593	0	2	5	5	12	255	At Risk
26174	26328	Eugene King	319	13	37519	0	2	5	5	12	255	At Risk
11186	11254	Alexis Hodges	385	11	37233	0	2	5	5	12	255	At Risk
40241	40504	David Dunn	446	11	37019	0	1	5	5	11	155	At Risk
32023	32218	Donna May	253	10	36837	0	2	5	5	12	255	At Risk
46555	46855	Tyler Foley	255	10	36804	0	2	5	5	12	255	At Risk

30830	31017	Sherry Phillips	256	11	36309	0	2	5	5	12	255	At Risk
34510	34720	Jeffrey Blanchard	534	11	36175	0	1	5	5	11	155	At Risk
9716	9775	Alan Jensen	380	11	36166	0	2	5	5	12	255	At Risk
43065	43344	Juan Smith	311	12	36083	0	2	5	5	12	255	At Risk
36735	36965	Trevor Davis	302	11	35844	0	2	5	5	12	255	At Risk
41886	42161	John Davis	268	13	35469	0	2	5	5	12	255	At Risk

===== DASHBOARD KPI TABLE =====

	KPI	Value
0	Total Customers	4.967300e+04
1	Total Transactions	2.500000e+05
2	Total Revenue	6.813427e+08
3	Average Order Value	2.725370e+03
4	Total Quantity Sold	7.497240e+05
5	Total Returns	1.007690e+05
6	Churn Rate (%)	1.995000e+01

=====
STEP 6 COMPLETED SUCCESSFULLY
=====

Visualization Output Files:

- ✓ age_group_visualization.csv
- ✓ category_visualization.csv
- ✓ customer_segment_visualization.csv
- ✓ dashboard_kpis.csv
- ✓ day_of_week_visualization.csv
- ✓ gender_visualization.csv
- ✓ hourly_purchase_visualization.csv
- ✓ master_visualization_summary.csv
- ✓ monthly_sales_visualization.csv
- ✓ payment_method_visualization.csv
- ✓ quantity_visualization.csv
- ✓ segment_churn_visualization.csv
- ✓ yearly_sales_visualization.csv

===== FINAL DASHBOARD KPIs =====

Total Customers	: 49,673
Total Transactions	: 250,000
Total Revenue	: 681,342,683.00
Average Order Value	: 2,725.37
Total Quantity Sold	: 749,724
Total Returns	: 100,769.0
Churn Rate	: 19.95%

Step 6 visualization and dashboard preparation completed.